



ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE  
TECHNOLOGIES

**RAPPORT DE STAGE D'ÉTÉ (STAGE INGÉNIEUR)**

**Option : Architecture des Réseaux et Technologies de l'Information  
et de la Communication (ArcTIC)**

---

## **Kubernetes Enterprise Platform (KEP)**

Conception, Déploiement, Sécurisation Zéro-Trust, Observabilité et  
Auto-guérison AIOps

---

*Elaboré par :*

**Farah BEN CHIKHA**

Élève Ingénieur ArcTIC

*Encadrant Académique :*

**Pr. Omar FITOURI**

Professeur Universitaire

Organisme d'Accueil : **École Supérieure Privée d'Ingénierie et de  
Technologies (ESPRIT)**

Année Universitaire : 2025 - 2026

# Remerciements

Je remercie chaleureusement Monsieur \*\*Omar FITOURI\*\*, mon encadrant académique, pour son suivi rigoureux, ses précieux conseils, son expertise technique, ses orientations pertinentes et son accompagnement tout au long de la réalisation de ce travail.

Je remercie également l'ensemble du corps enseignant et administratif de l'\*\*École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT)\*\* pour la qualité de la formation dispensée durant ces années d'études au sein de l'option \*\*ArcTIC\*\* (Architecture des Réseaux et Technologies de l'Information et de la Communication).

Mes remerciements vont aussi à toute l'équipe d'ESPRIT pour m'avoir offert l'opportunité d'effectuer ce stage et de travailler sur un projet aussi enrichissant.

Enfin, je remercie chaleureusement mes parents pour leur soutien indéfectible, leurs encouragements constants et leur confiance tout au long de mon parcours académique.

# Présentation de l'Organisme d'Accueil

## L'École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT)

Fondée en 2003, l'École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT)\*\* est une institution universitaire privée majeure en Tunisie, accréditée à l'échelle internationale (notamment par la commission \*\*EUR-ACE\*\* et le réseau \*\*CDIO\*\*). ESPRIT se distingue par son modèle pédagogique novateur axé sur l'Apprentissage Par Projets / Problèmes (APP), formant des ingénieurs hautement qualifiés et immédiatement opérationnels sur le marché mondial.



FIGURE 1 – Logo officiel d'ESPRIT

## L'Option ArcTIC (Architecture des Réseaux et TIC)

L'option \*\*ArcTIC\*\* (Architecture des Réseaux et Technologies de l'Information et de la Communication) forme des ingénieurs spécialistes des infrastructures numériques modernes, du Cloud Computing (Kubernetes, DevOps, GitOps), de la cybersécurité Zéro-Trust et de l'IA appliquée aux réseaux (AIOps). C'est dans ce cadre académique que s'inscrit la conception et le déploiement de la plateforme \*\*KEP\*\*.

# Table des matières

|                                                                                                                    |           |
|--------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Remerciements</b>                                                                                               | <b>1</b>  |
| <b>Présentation de l'Organisme d'Accueil</b>                                                                       | <b>2</b>  |
| <b>Introduction Générale</b>                                                                                       | <b>5</b>  |
| <b>1 Présentation du Projet, Analyse des Besoins &amp; Choix Technologiques</b>                                    | <b>6</b>  |
| 1.1 Analyse des Besoins . . . . .                                                                                  | 6         |
| 1.1.1 Besoins Fonctionnels (BF) . . . . .                                                                          | 6         |
| 1.1.2 Besoins Non-Fonctionnels (BNF) . . . . .                                                                     | 6         |
| 1.2 Justification des Choix Technologiques . . . . .                                                               | 7         |
| 1.3 Catalogue des Outils Retenus et Emplacements d'Utilisation . . . . .                                           | 9         |
| 1.4 Planification des Sprints Agile Scrum (8 Semaines / 2 Mois) . . . . .                                          | 10        |
| <b>2 Architecture Système &amp; Provisionnement IaaS / Configuration (Terraform &amp; Ansible)</b>                 | <b>11</b> |
| 2.1 Architecture Détaillée de la Plateforme (3 VMs) . . . . .                                                      | 11        |
| 2.2 Pipeline et Fonctionnement Opérationnel de l'Architecture . . . . .                                            | 12        |
| 2.2.1 Phase 1 — Entrées, Accès Utilisateurs et Administration . . . . .                                            | 12        |
| 2.2.2 Phase 2 — Orchestration, GitOps et Intelligence (Nœud 'k8s-master' — IP 192.168.233.188) . . . . .           | 13        |
| 2.2.3 Phase 3 — Exécution Applicative et Injection de Secrets (Nœud 'k8s-worker' — IP 192.168.233.190) . . . . .   | 13        |
| 2.2.4 Phase 4 — Sécurité Zéro-Trust et Observabilité Centralisée (Nœud 'elk-vault' — IP 192.168.233.189) . . . . . | 13        |
| 2.2.5 Phase 5 — Intégration avec les Services et Registres Externes . . . . .                                      | 14        |
| 2.3 Provisionnement IaaS avec Terraform . . . . .                                                                  | 14        |
| 2.4 Industrialisation de la Configuration avec Ansible . . . . .                                                   | 15        |
| 2.5 Initialisation du Cluster Kubernetes . . . . .                                                                 | 16        |
| <b>3 Sécurité Zéro-Trust &amp; Observabilité Centralisée (Vault &amp; ELK)</b>                                     | <b>17</b> |
| 3.1 Sécurisation Zéro-Trust avec HashiCorp Vault . . . . .                                                         | 17        |
| 3.1.1 Configuration de l'Authentification Kubernetes . . . . .                                                     | 17        |

|          |                                                                                  |           |
|----------|----------------------------------------------------------------------------------|-----------|
| 3.1.2    | Injection par Conteneur Sidecar en RAM Volatile ( <code>tmpfs</code> ) . . . . . | 17        |
| 3.1.3    | Résolution du Routage Réseau Virtuel Retour . . . . .                            | 17        |
| 3.2      | Observabilité Centralisée avec la Pile ELK . . . . .                             | 18        |
| 3.2.1    | Résolution du Dépannage ‘ContainerCreating’ . . . . .                            | 18        |
| <b>4</b> | <b>GitOps, Alerting Temps Réel &amp; Auto-guérison AIOps</b>                     | <b>20</b> |
| 4.1      | Déploiement Applicatif WokMaster & Continuous Delivery GitOps . . . . .          | 20        |
| 4.2      | Alerting Temps Réel avec Kubewatch et Discord . . . . .                          | 21        |
| 4.3      | Diagnostic & Auto-guérison AIOps (K8sGPT & Agent MCP Custom) . . . . .           | 22        |
| 4.3.1    | Diagnostic IA avec K8sGPT v0.3.40 . . . . .                                      | 22        |
| 4.3.2    | Auto-guérison Autonome avec l’Agent MCP Custom . . . . .                         | 22        |
|          | <b>Conclusion Générale et Perspectives</b>                                       | <b>23</b> |
| <b>A</b> | <b>Annexes</b>                                                                   | <b>24</b> |
| A.1      | Annexe A : Fichier d’inventaire Ansible (‘hosts.ini’) . . . . .                  | 24        |
| A.2      | Annexe B : Code complet du Serveur MCP (‘mcp_k8s_server.py’) . . . . .           | 24        |
| A.3      | Annexe C : Manifeste GitOps ArgoCD (‘argocd-app.yaml’) . . . . .                 | 26        |
|          | <b>Webographie et Ressources</b>                                                 | <b>28</b> |

# Introduction Générale

## Contexte et Enjeux

Dans le domaine du Cloud Native, du DevOps et des architectures réseaux modernes, la gestion des infrastructures d'entreprise exige des niveaux élevés d'automatisation, de sécurité et de résilience. Les approches traditionnelles de déploiement manuel souffrent fréquemment de dérives de configuration (\*configuration drift\*), de fuites d'identifiants sensibles en clair et de temps d'arrêt prolongés lors des pannes.

Ce rapport retrace les travaux menés lors de mon stage ingénieur d'été au sein d'\*\*ESPRIT\*\*, sous l'encadrement du \*\*Pr. Omar FITOURI\*\*, pour concevoir et implémenter la plateforme \*\*Kubernetes Enterprise Platform (KEP)\*\*. KEP est une infrastructure d'hébergement d'entreprise basée sur \*\*Kubernetes\*\*, destinée à orchestrer l'application métier \*\*WokMaster\*\* (backend Spring Boot, frontend Angular, MySQL 8.0).

---

# Chapitre 1

## Présentation du Projet, Analyse des Besoins & Choix Technologiques

### 1.1 Analyse des Besoins

#### 1.1.1 Besoins Fonctionnels (BF)

1. **\*\*BF-01** — Provisionnement IaaS automatisé :**\*\*** Cloner et instancier automatiquement 3 VMs sous VMware via du code déclaratif (IaC).
2. **\*\*BF-02** — Configuration logicielle idempotente :**\*\*** Automatiser la préparation système, l'installation du moteur de conteneurs et du cluster K8s sans intervention manuelle.
3. **\*\*BF-03** — Orchestration de conteneurs :**\*\*** Assurer l'exécution, le découpage et le routage des conteneurs applicatifs WokMaster (Frontend, Backend, MySQL).
4. **\*\*BF-04** — Sécurité Zéro-Trust des secrets :**\*\*** Interdire le stockage des mots de passe sur disque et les injecter dynamiquement en mémoire RAM volatile.
5. **\*\*BF-05** — Observabilité centralisée :**\*\*** Collecter et consolider l'intégralité des journaux applicatifs (\*logs\*) et des métriques matérielles sur un tableau de bord unique.
6. **\*\*BF-06** — Continuous Delivery & GitOps :**\*\*** Garantir la synchronisation automatique de l'état du cluster avec le dépôt GitHub et éliminer la dérive de configuration.
7. **\*\*BF-07** — Alerting temps réel :**\*\*** Notifier instantanément l'équipe d'exploitation sur Discord en cas de crash de conteneur.
8. **\*\*BF-08** — Diagnostic IA & Auto-guérison AIOps :**\*\*** Analyser les pannes en langage naturel et exécuter des remédiations autonomes sécurisées sous contrôle humain.

#### 1.1.2 Besoins Non-Fonctionnels (BNF)

1. **\*\*BNF-01** — Sécurité renforcée (Zéro-Trust) :**\*\*** Chiffrement fort AES-256 des variables sensibles et isolation des secrets en mémoire RAM volatile (`tmpfs`).

2. **\*\*BNF-02 — Haute Disponibilité & Isolation : \*\*** Séparation physique stricte des rôles (Control Plane, Worker, et serveur externe de Sécurité/Observabilité).
3. **\*\*BNF-03 — Idempotence & Reproductibilité : \*\*** Capacité de rejouer les déploiements à l'infini avec un résultat identique et sans effets de bord.
4. **\*\*BNF-04 — Faible Latence d'Alerte : \*\*** Délai de notification des pannes sur Discord inférieur à 2 secondes.
5. **\*\*BNF-05 — Contrôle Humain (Human-in-the-Loop) : \*\*** Garantie qu'aucune action destructive d'auto-rémediation IA ne s'exécute sans l'accord explicite d'un opérateur.

## 1.2 Justification des Choix Technologiques

Le Tableau [1.1](#) récapitule les raisons techniques qui ont motivé la sélection de notre chaîne d'outils par rapport aux alternatives du marché :

TABLE 1.1 – Justification comparative des choix technologiques de la plateforme KEP

| Domaine       | Outil Retenu               | Alternatives           | Justification du Choix Retenu                                                                                                                              |
|---------------|----------------------------|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IaC           | <b>**Terraform**</b>       | Vagrant, Scripts Shell | Gestion d'état déclaratif ( <code>.tfstate</code> ), intégration native API REST VMware, reproductibilité entreprise.                                      |
| Configuration | <b>**Ansible**</b>         | Puppet, Chef           | Architecture <i>*agentless*</i> (SSH natif), syntaxe YAML accessible, coffre-fort d'identifiants <b>**Ansible Vault (AES-256)**</b> .                      |
| Orchestration | <b>**Kubernetes**</b>      | Docker Swarm           | Standard CNCF incontournable, richesse de l'écosystème (Operators, Sidecars, CRDs), auto-scaling et auto-healing natifs.                                   |
| Sécurité      | <b>**HashiCorp Vault**</b> | K8s Secrets            | Les K8s Secrets sont seulement encodés en Base64. Vault offre un chiffrement fort, l'authentification dynamique et l'injection en RAM <code>tmpfs</code> . |
| Observabilité | <b>**Pile ELK**</b>        | Prometheus / Grafana   | Indexation textuelle distribuée avancée des journaux d'audit et corrélation puissante des logs sur Kibana.                                                 |
| GitOps        | <b>**ArgoCD**</b>          | Jenkins, GitLab CI     | Modèle <i>*Pull-based*</i> empêchant la dérive de configuration ( <i>*Self-healing*</i> natif) au lieu d'un pipeline <i>*Push*</i> aveugle.                |
| AIOps / Agent | <b>**MCP + Gemini**</b>    | Scripts Shell / Cron   | Raisonnement contextualisé LLM avec barrière de sécurité <i>*Human-in-the-Loop*</i> au lieu de règles rigides sans compréhension de la cause racine.       |

## 1.3 Catalogue des Outils Retenus et Emplacements d'Utilisation

1. **\*\*Terraform : \*\*** Provisionnement IaaS déclenché depuis le poste d'administration Windows vers l'API VMware (Port 8697).
2. **\*\*Ansible & Ansible Vault : \*\*** Industrialisation de la configuration exécutée depuis le Master vers les 3 VMs via SSH (Port 22).
3. **\*\*Kubernetes v1.30 (kubeadm, containerd, Calico) : \*\*** Orchestration de conteneurs sur **k8s-master** et **k8s-worker**.
4. **\*\*HashiCorp Vault : \*\*** Moteur de sécurité Zéro-Trust hébergé sur la VM **elk-vault** (Port 8200 HTTPS).
5. **\*\*Elasticsearch 8.x & Kibana : \*\*** Moteur de recherche et d'analyse visuelle des logs hébergé sur **elk-vault** (Ports 9200 et 5601).
6. **\*\*Filebeat & Metricbeat : \*\*** Agents de collecte de logs et de télémétrie déployés en DaemonSets sur **k8s-worker**.
7. **\*\*ArgoCD : \*\*** Moteur GitOps déployé sur **k8s-master** et exposé sur le port NodePort 30086.
8. **\*\*Kubewatch : \*\*** Agent de surveillance réseau déployé sur **k8s-master** connecté au Webhook Discord.
9. **\*\*K8sGPT v0.3.40 : \*\*** Outil de diagnostic rapide basé sur Google Gemini installé sur **k8s-master**.
10. **\*\*Agent MCP Custom (Python + Gemini 3.6-Flash) : \*\*** Agent d'auto-guérison AIOps exécuté en tâche de fond sur **k8s-master**.

## 1.4 Planification des Sprints Agile Scrum (8 Semaines / 2 Mois)

Les travaux ont été organisés selon la méthodologie **\*\*Agile Scrum\*\*** sur 4 Sprints de 2 semaines :

TABLE 1.2 – Découpage du projet KEP en Sprints Agile Scrum (8 semaines)

| Sprint          | Période    | Livrables Majeurs                                                                               |
|-----------------|------------|-------------------------------------------------------------------------------------------------|
| <b>Sprint 1</b> | Sem. 1 & 2 | Étude des besoins, architecture 3 VMs, scripts PowerShell API et Terraform IaC.                 |
| <b>Sprint 2</b> | Sem. 3 & 4 | Configuration Ansible, installation <code>containerd</code> , initialisation K8s avec Calico.   |
| <b>Sprint 3</b> | Sem. 5 & 6 | Sécurité HashiCorp Vault ( <code>tmpfs</code> RAM) et observabilité ELK (Filebeat, Metricbeat). |
| <b>Sprint 4</b> | Sem. 7 & 8 | GitOps ArgoCD, Alerting Discord (Kubewatch), K8sGPT et Agent MCP Gemini.                        |

# Chapitre 2

## Architecture Système & Provisionnement IaaS / Configuration (Terraform & Ansible)

### 2.1 Architecture Détaillée de la Plateforme (3 VMs)

La plateforme repose sur trois machines virtuelles isolées sous VMware Workstation Pro, interconnectées sur un réseau privé virtuel (192.168.233.0/24) :

TABLE 2.1 – Dimensionnement matériel réel des 3 machines virtuelles KEP

| Machine Virtuelle | Adresse IP      | vCPU   | Mémoire RAM       | Services Hébergés              |
|-------------------|-----------------|--------|-------------------|--------------------------------|
| k8s-master        | 192.168.233.188 | 2 vCPU | 2300 Mo ( 2.3 Go) | Control Plane K8s, ArgoCD, K   |
| k8s-worker        | 192.168.233.190 | 1 vCPU | 4096 Mo (4 Go)    | Angular, Spring Boot, MySQL    |
| elk-vault         | 192.168.233.189 | 2 vCPU | 3072 Mo (3 Go)    | HashiCorp Vault, Elasticsearch |

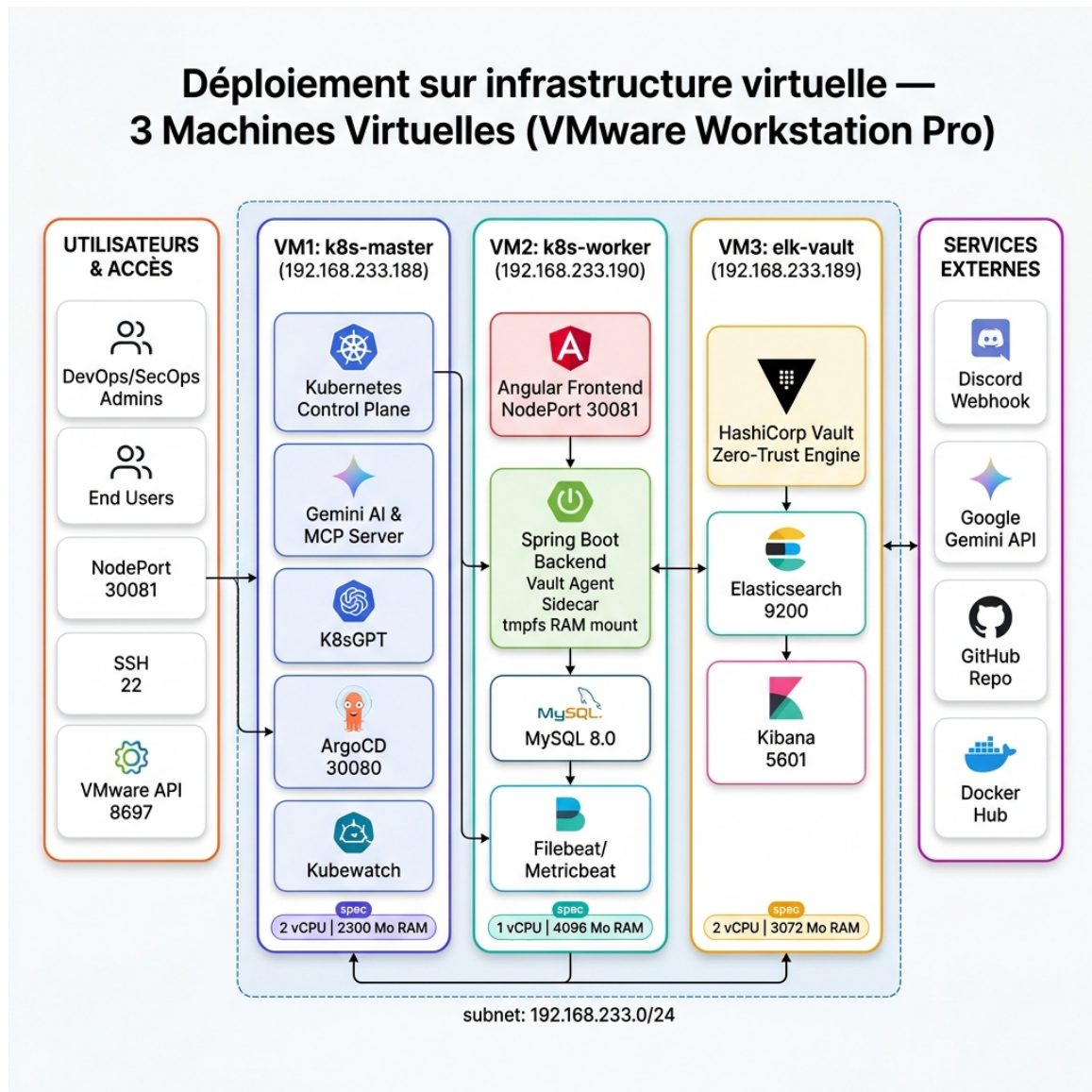


FIGURE 2.1 – Schéma d’architecture physique et applicative de la plateforme KEP sur 3 VMs

## 2.2 Pipeline et Fonctionnement Opérationnel de l’Architecture

Le fonctionnement global de la plateforme KEP repose sur une interaction en chaîne découpée en **5 phases opérationnelles fluides** :

### 2.2.1 Phase 1 — Entrées, Accès Utilisateurs et Administration

- **DevOps / SecOps Admins** : Administrent les machines virtuelles via le protocole sécurisé **SSH (Port 22)** et déclenchent le provisionnement IaaS via l’**API REST VMware (Port 8697)**.

- **Utilisateurs Finaux (\*End Users\*)** : Accèdent à l'interface web de l'application métier WokMaster à travers le port exposé **NodePort 30081** du nœud Worker.

### 2.2.2 Phase 2 — Orchestration, GitOps et Intelligence (Nœud ‘k8s-master’ — IP 192.168.233.188)

- **Kubernetes Control Plane** : Reçoit les ordres de déploiement, gère l'état du cluster, planifie les conteneurs (\*scheduling\*) et assure la santé générale des nœuds.
- **ArgoCD (Port 30086)** : Scrute le dépôt **GitHub** en continu (\*Pull-based\*). Dès qu'un manifeste est modifié sur Git, ArgoCD applique automatiquement la mise à jour dans le cluster et annule toute modification manuelle non autorisée (\*self-healing\*).
- **Kubewatch** : Surveille les événements du cluster K8s et pousse immédiatement les alertes de crash vers le **Webhook Discord**.
- **Diagnostic IA & Serveur MCP Gemini** : **K8sGPT** et le **Serveur MCP (Model Context Protocol)** interceptent les erreurs complexes, interrogent l'API **Google Gemini 3.6-Flash** pour générer un diagnostic en langage naturel et proposent une remédiation autonome sécurisée.

### 2.2.3 Phase 3 — Exécution Applicative et Injection de Secrets (Nœud ‘k8s-worker’ — IP 192.168.233.190)

- **Microservices WokMaster** : Le frontend **Angular** communique avec le backend **Spring Boot API**, qui s'appuie sur la base de données **MySQL 8.0** (données persistées dans `/mnt/data/mysql`).
- **Sidecar Vault Agent & Mémoire Volatile RAM (tmpfs)** : À chaque démarrage du backend, un conteneur compagnon (\*sidecar\*) s'intercale, s'authentifie auprès de Vault, récupère le mot de passe MySQL et le dépose dans un volume en mémoire vive RAM (`/vault/secrets/config`). Aucun secret n'est écrit sur disque.
- **Agents de collecte Filebeat & Metricbeat** : Capturent les logs de conteneurs (`/var/log/containers/*.log`) et les métriques de consommation (vCPU/RAM) et les transmettent à Elasticsearch.

### 2.2.4 Phase 4 — Sécurité Zéro-Trust et Observabilité Centralisée (Nœud ‘elk-vault’ — IP 192.168.233.189)

- **HashiCorp Vault Engine** : Valide le jeton JWT du pod via le rôle `system:auth-delegator` de Kubernetes et distribue les identifiants de base de données de façon chiffrée.
- **Elasticsearch 8.x (Port 9200)** : Moteur de stockage et d'indexation distribué recevant les flux de données de Filebeat et Metricbeat.

- **\*\*Kibana (Port 5601) :\*\*** Interface de restitution visuelle permettant aux administrateurs de rechercher, filtrer et analyser l'ensemble des journaux d'audit et métriques en temps réel.

## 2.2.5 Phase 5 — Intégration avec les Services et Registres Externes

- **\*\*Docker Hub :\*\*** Registre d'images conteneurisées hébergeant les images de production `farahbenchikha/spring-backend:v1` et `farahbenchikha/angular-frontend:v1`.
- **\*\*GitHub Repo :\*\*** Source unique de vérité (\*Single Source of Truth\*) hébergeant le code source et les manifestes YAML Kubernetes.
- **\*\*Discord Webhook & Google Gemini API :\*\*** Canaux externes assurant respectivement l'envoi d'alertes instantanées sur smartphone/PC et la puissance de calcul IA pour l'auto-guérison AIOps.

## 2.3 Provisionnement IaaS avec Terraform

Le provisionnement des machines virtuelles est entièrement automatisé. Le service d'API REST VMware Workstation (`vmrest.exe`) est démarré sur le port 8697. Terraform interroge cet API pour cloner l'image de base `ubuntu-base` :

```
1 cd "C:\Program Files (x86)\VMware\VMware Workstation" && .\vmrest.exe -p 8697
2 cd terraform/ && terraform init && terraform apply -parallelism=1
```

Listing 2.1 – Démarrage de l'API VMware et exécution de Terraform

L'option `-parallelism=1` sérialise la création des VMs afin d'éviter les verrous d'accès concurrents sur le disque hôte.

```
PS C:\Users\farah\kubernetes-enterprise-platform\terraform> terraform apply -parallelism=3

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# vmworkstation_virtual_machine.elk-vault will be created
+ resource "vmworkstation_virtual_machine" "elk-vault" {
  + denomination = "elk-vault"
  + id           = (known after apply)
  + ip          = "0.0.0.0/0"
  + memory      = 3872
  + path        = "C:\\Users\\farah\\Virtual Machines\\elk-vault\\elk-vault.vmx"
  + processors   = 2
  + sourceid    = "JU1H89H67D6MTF09GQ012VIU2PJT148E"
  + state       = "on"
}

# vmworkstation_virtual_machine.k8s-master will be created
+ resource "vmworkstation_virtual_machine" "k8s-master" {
  + denomination = "k8s-master"
  + id           = (known after apply)
  + ip          = "0.0.0.0/0"
  + memory      = 2048
  + path        = "C:\\Users\\farah\\Virtual Machines\\k8s-master\\k8s-master.vmx"
  + processors   = 2
  + sourceid    = "JU1H89H67D6MTF09GQ012VIU2PJT148E"
  + state       = "on"
}

# vmworkstation_virtual_machine.k8s-worker will be created
+ resource "vmworkstation_virtual_machine" "k8s-worker" {
  + denomination = "k8s-worker"
  + id           = (known after apply)
  + ip          = "0.0.0.0/0"
  + memory      = 2048
  + path        = "C:\\Users\\farah\\Virtual Machines\\k8s-worker\\k8s-worker.vmx"
  + processors   = 1
  + sourceid    = "JU1H89H67D6MTF09GQ012VIU2PJT148E"
  + state       = "on"
}

Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

vmworkstation_virtual_machine.k8s-master: Creating...
vmworkstation_virtual_machine.k8s-master: Still creating... [00m10s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [00m20s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [00m30s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [00m40s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [00m50s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [01m00s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [01m10s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [01m20s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [01m30s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [01m40s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [01m50s elapsed]
vmworkstation_virtual_machine.k8s-master: Still creating... [02m00s elapsed]
```

FIGURE 2.2 – Provisionnement séquentiel des 3 VMs via terraform apply

## 2.4 Industrialisation de la Configuration avec Ansible

Afin de remplacer les configurations manuelles par des tâches automatisées et idempotentes, un inventaire `hosts.ini` et des playbooks déclaratifs ont été rédigés.

Les identifiants SSH et mots de passe `sudo` sont protégés via **\*\*Ansible Vault (AES-256)\*\*** dans le fichier `secrets.yml` :

```
1 # secrets.yml (Chiffre AES-256 via : ansible-vault encrypt secrets.yml)
2 vault_ssh_pass: "farah"
3 vault_become_pass: "farah"
```

Listing 2.2 – Chiffrement des variables SSH avec Ansible Vault

Le script Shell d'initialisation des nœuds (`install_k8s.sh`) automatise : 1. La désactivation permanente du swap (`swapoff -a` et suppression dans `/etc/fstab`). 2. Le chargement des modules du noyau Linux (`overlay` et `br_netfilter`). 3. La configuration de `containerd` avec le gestionnaire de ressources `SystemdCgroup=true`. 4. L'installation des paquets `kubelet`, `kubeadm`, `kubectl` v1.30 avec blocage des mises à jour (`apt-mark hold`).

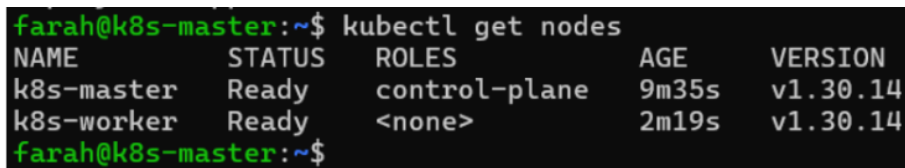
## 2.5 Initialisation du Cluster Kubernetes

Le Control Plane est initialisé sur le Master (192.168.233.188) :

```
1 sudo kubeadm init --pod-network-cidr=192.168.0.0/16 --apiserver-advertise-  
   address=192.168.233.188  
2 kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3  
   .26.1/manifests/calico.yaml
```

Listing 2.3 – Initialisation du Master et déploiement du réseau Calico CNI

Le nœud Worker est ensuite raccordé via la commande `kubeadm join`.



```
farah@k8s-master:~$ kubectl get nodes  
NAME           STATUS    ROLES          AGE      VERSION  
k8s-master     Ready     control-plane   9m35s    v1.30.14  
k8s-worker     Ready     <none>          2m19s    v1.30.14  
farah@k8s-master:~$
```

FIGURE 2.3 – Validation du statut opérationnel des nœuds Master et Worker (Ready)

# Chapitre 3

## Sécurité Zéro-Trust & Observabilité Centralisée (Vault & ELK)

### 3.1 Sécurisation Zéro-Trust avec HashiCorp Vault

Afin d'interdire tout stockage de mot de passe en clair dans les fichiers du projet ou sur le disque dur, HashiCorp Vault est déployé sur la VM `elk-vault` (192.168.233.189).

#### 3.1.1 Configuration de l'Authentification Kubernetes

Un compte de service dédié (`vault-auth`) est créé sur le Master avec le rôle `system:auth-delegator`. Ce rôle autorise Vault à interroger l'API Server de Kubernetes via des requêtes de type `TokenReview` pour valider l'identité des conteneurs.

#### 3.1.2 Injection par Conteneur Sidecar en RAM Volatile (`tmpfs`)

Sur le Master, le contrôleur `**Vault Agent Injector**` est déployé via Helm. Lors du démarrage d'un pod applicatif portant les annotations Vault : 1. L'injecteur intercepte la requête et monte un conteneur sidecar `vault-agent-init`. 2. Le sidecar s'authentifie auprès de Vault, récupère les identifiants MySQL et les dépose dans un volume volatile mounted en RAM (`/vault/secrets/config`). 3. L'application lit le secret depuis la RAM. Dès la destruction du pod, les secrets sont immédiatement effacés.

```
1 kubectl exec -it deployment/vault-demo -c app -- df -h /vault/secrets
2 # R sultat: tmpfs 64M 4.0K 64M 1% /vault/secrets
```

Listing 3.1 – Vérification du stockage des secrets exclusivement en mémoire RAM volatile

#### 3.1.3 Résolution du Routage Réseau Virtuel Retour

Les pods s'exécutant sur le Worker (192.168.233.190) dans le sous-réseau virtuel Calico (192.168.0.0/16), la VM `elk-vault` ne connaissait pas le chemin retour. Une `**route`

statique\*\* a été configurée sur elk-vault :

```
1 sudo ip route add 192.168.0.0/16 via 192.168.233.190
```

[illegible]

FIGURE 3.1 – Validation par les logs applicatifs de la lecture du secret injecté depuis Vault en RAM

### 3.2 Observabilité Centralisée avec la Pile ELK

Les agents **Filebeat** (collecte de logs `/var/log/containers/*.log`) et **Metricbeat** (télémétrie CPU/RAM) sont déployés sous forme de DaemonSets Helm.

### 3.2.1 Résolution du Dépannage ‘ContainerCreating’

Le pod Filebeat restait initialement bloqué au statut `ContainerCreating` en raison du manque des secrets TLS par défaut de la charte Helm. La création de secrets fictifs a débloquent le montage `kubelet` :

```
1 kubectl create secret generic elasticsearch-master-certs --from-literal=
    dummy=data
2 kubectl create secret generic elasticsearch-master-credentials --from-
    literal=username=elastic --from-literal=password=dummy
```

L'inspection des logs Filebeat confirme l'envoi réussi des données vers Elasticsearch : "acked": 4096 (4096 lignes de logs acquittées).

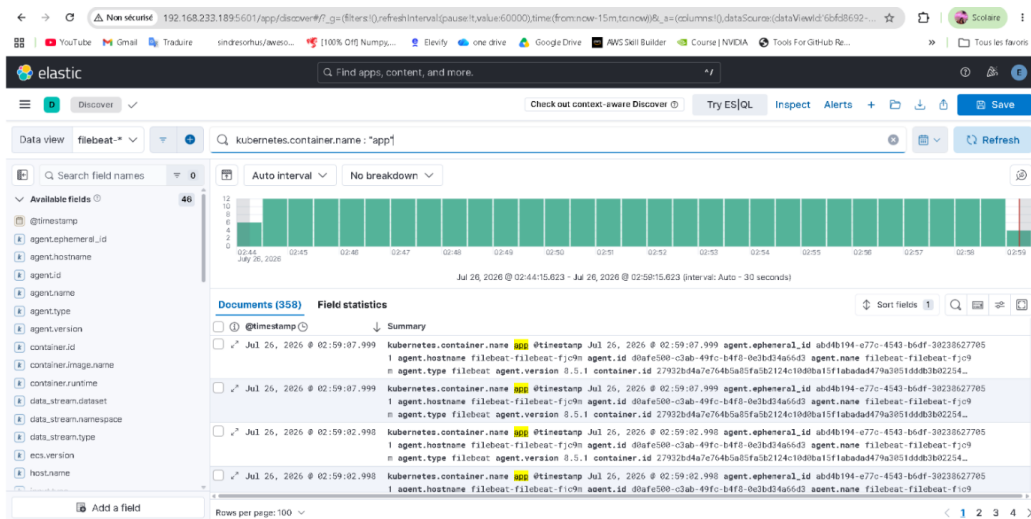


FIGURE 3.2 – Visualisation et recherche des journaux de conteneurs en temps réel sur Kibana

# Chapitre 4

## GitOps, Alerting Temps Réel & Auto-guérison AIOps

### 4.1 Déploiement Applicatif WokMaster & Continuous Delivery GitOps

L'application métier **WokMaster** est containerisée via des **Multi-stage Builds** Docker (Backend Spring Boot Java 17 et Frontend Angular 17 sur Nginx Alpine).

Pour appliquer la philosophie GitOps, **ArgoCD** est déployé sur le Master.

**Résolution de problème :** Le manifeste officiel dépassant la limite de 256 Ko d'annotation Kubernetes, le déploiement a été effectué par **Server-Side Apply** (`kubectl apply -server-side -force-conflicts`).

ArgoCD scrute le dépôt GitHub en continu. Si une modification manuelle a lieu dans le cluster, ArgoCD écrase l'écart pour rétablir la version déclarée sur GitHub (\*self-healing\* anti-dérive).

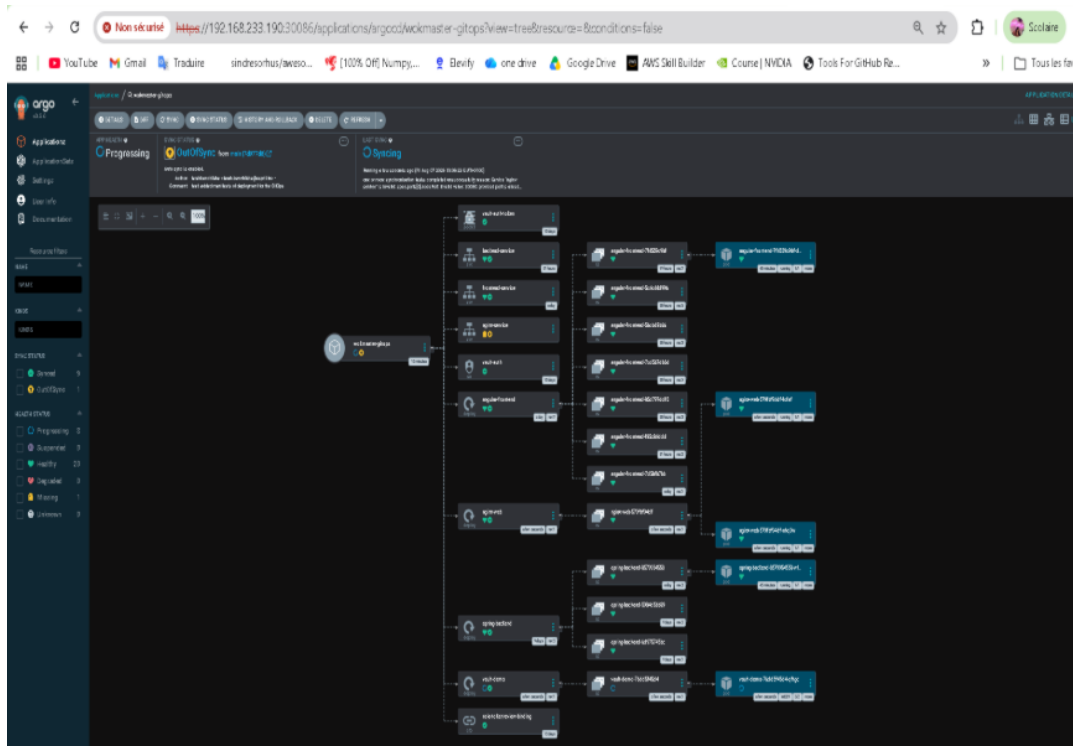


FIGURE 4.1 – Arbre visuel de synchronisation GitOps de l'application WokMaster dans ArgoCD

## 4.2 Alerting Temps Réel avec Kubewatch et Discord

L'agent léger **Kubewatch** surveille les événements du cluster. La résolution DNS de CoreDNS a été mise à jour vers 8.8.8.8 pour permettre la traversée des alertes HTTPS vers Discord. Dès qu'un pod entre en défaillance (**CrashLoopBackOff** ou **ImagePullBackOff**), une notification instantanée est envoyée sur le canal Discord de l'équipe.

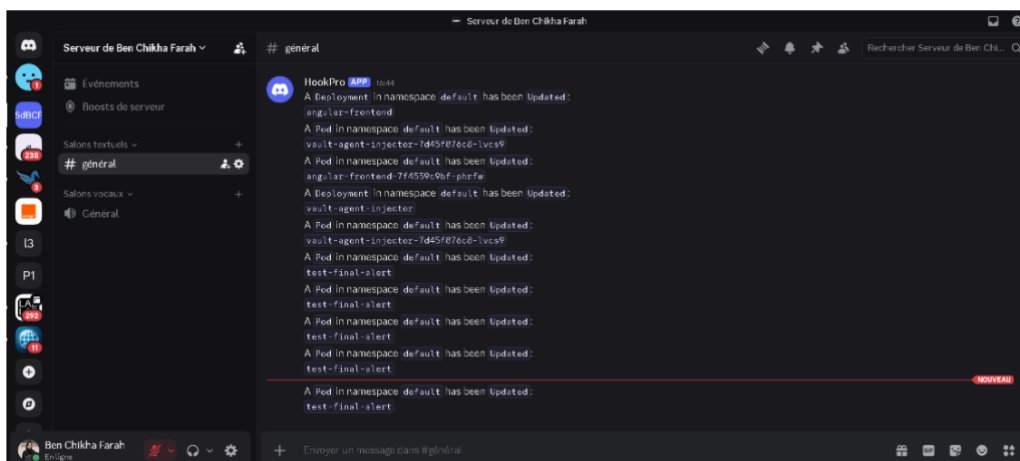


FIGURE 4.2 – Notification d'incident reçue en temps réel sur le canal Discord

### 4.3 Diagnostic & Auto-guérison AIOps (K8sGPT & Agent MCP Custom)

### 4.3.1 Diagnostic IA avec K8sGPT v0.3.40

L'outil CNCF K8sGPT est configuré avec l'API Google Gemini 3.6-Flash. Lors de la simulation d'un crash de pod, la commande `k8sgpt analyze -explain -backend google` retourne en 3 secondes un rapport explicatif clair avec la solution corrective.

### 4.3.2 Auto-guérison Autonome avec l'Agent MCP Custom

Un agent d'auto-guérison basé sur le protocole **\*\*Model Context Protocol (MCP)\*\*** a été développé en Python :

- **\*\*Serveur MCP (mcp\_k8s\_server.py) :\*\*** Expose 3 outils JSON-RPC sur stdio (`list_pods`, `get_pod_logs`, `restart_deployment`).
- **\*\*Client Agent (mcp\_agent\_run.py) :\*\*** Orchestre les requêtes avec Gemini 3.6-Flash et intègre une barrière de sécurité **\*\*Human-in-the-Loop (HITL)\*\***.

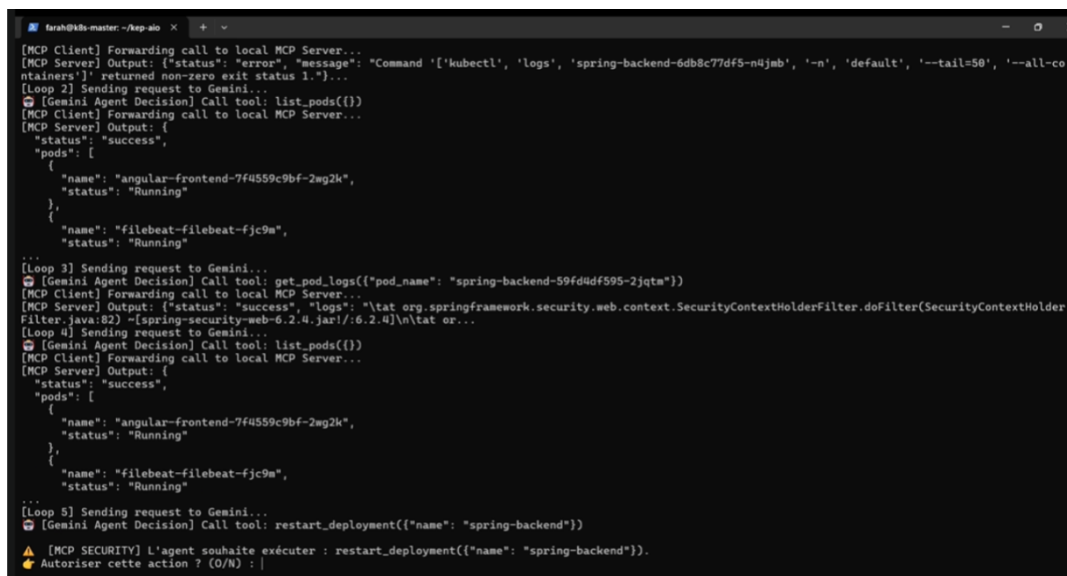


FIGURE 4.3 – Interception de sécurité de l’agent MCP sollicitant la confirmation de l’opérateur ([MCP SECURITY] (O/N))

**Scénario de démonstration :** Vault est verrouillé volontairement (`vault operator seal`), entraînant un crash du backend. Kubewatch alerte sur Discord. L’agent MCP analyse les logs, identifie que Vault est scellé, demande l’autorisation de redémarrage à l’opérateur (O/N), puis relance le déploiement après le déverrouillage de Vault. Le pod repasse au vert en 20 secondes.

# Conclusion Générale et Perspectives

## Conclusion

Ce stage d'été (stage ingénieur) effectué au sein d'**ESPRIT** dans le cadre de l'option **ArcTIC** sous la direction du **Pr. Omar FITOURI** a permis de mener à terme la conception et la mise en production de la plateforme d'entreprise **Kubernetes Enterprise Platform (KEP)**.

Ce projet démontre qu'il est possible d'allier une sécurité Zéro-Trust stricte (secrets injectés uniquement en mémoire vive RAM `tmpfs`), une automatisation intégrale par le code (Terraform, Ansible, GitOps ArgoCD), une observabilité totale (ELK) et une résilience moderne propulsée par l'Intelligence Artificielle (K8sGPT, Agent MCP Gemini).

## Perspectives

1. **Auto-scaling prédictif** : Prédire les pics de charge via l'IA pour ajuster proactivement le nombre de répliques.
2. **SecOps avancé** : Intégrer l'outil Trivy dans le serveur MCP pour scanner la vulnérabilité des images Docker avant tout redémarrage autonome.

---

# Annexe A

## Annexes

### A.1 Annexe A : Fichier d’inventaire Ansible (‘hosts.ini’)

```
1 [master]
2 k8s-master ansible_host=192.168.233.188
3
4 [worker]
5 k8s-worker ansible_host=192.168.233.190
6
7 [elk]
8 elk-vault ansible_host=192.168.233.189
9
10 [all:vars]
11 ansible_user=farah
12 ansible_ssh_pass="{{vault_ssh_pass}}"
13 ansible_become_pass="{{vault_become_pass}}"
14 ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

Listing A.1 – Fichier d’inventaire Ansible du cluster KEP

### A.2 Annexe B : Code complet du Serveur MCP (‘mcp<sub>k8s</sub>server.py’)

```
1 import sys, json, subprocess
2
3 def list_pods(namespace="default"):
4     try:
5         cmd = ["kubectl", "get", "pods", "-n", namespace, "-o", "json"]
6         res = subprocess.run(cmd, capture_output=True, text=True, check=
7             True)
8         data = json.loads(res.stdout)
9         pods = []
10        for item in data.get("items", []):
11            name = item["metadata"]["name"]
```

```

11         status = item["status"]["phase"]
12         container_statuses = item["status"].get("containerStatuses",
13         [])
14         for c_status in container_statuses:
15             state = c_status.get("state", {})
16             for state_name, state_details in state.items():
17                 if state_name in ["waiting", "terminated"]:
18                     status = state_details.get("reason", status)
19                     pods.append({"name": name, "status": status})
20             return json.dumps({"status": "success", "pods": pods})
21         except Exception as e:
22             return json.dumps({"status": "error", "message": str(e)})
23
24 def get_pod_logs(pod_name, namespace="default"):
25     try:
26         cmd = ["kubectl", "logs", pod_name, "-n", namespace, "--tail=50"]
27         res = subprocess.run(cmd, capture_output=True, text=True, check=
28         True)
29         return json.dumps({"status": "success", "logs": res.stdout})
30     except Exception as e:
31         return json.dumps({"status": "error", "message": str(e)})
32
33 def restart_deployment(name, namespace="default"):
34     try:
35         cmd = ["kubectl", "rollout", "restart", f"deployment/{name}", "-n"
36         , namespace]
37         res = subprocess.run(cmd, capture_output=True, text=True, check=
38         True)
39         return json.dumps({"status": "success", "message": f"Rollout_
40         restart_triggered_for_deployment/{name}"})
41     except Exception as e:
42         return json.dumps({"status": "error", "message": str(e)})
43
44
45 TOOLS = {
46     "list_pods": list_pods,
47     "get_pod_logs": get_pod_logs,
48     "restart_deployment": restart_deployment
49 }
50
51
52 def main():
53     print("Kubernetes_MCP_Server_initialized.", file=sys.stderr)
54     while True:
55         try:
56             line = sys.stdin.readline()
57             if not line: break
58             req = json.loads(line)
59             method = req.get("method")
60             params = req.get("params", {})

```

```

54         req_id = req.get("id")
55
56         if method == "tools/list":
57             resp = {"jsonrpc": "2.0", "id": req_id, "result": {"tools"
58 : [
59                 {"name": "list_pods", "description": "List all pods.",
60                  "inputSchema": {"type": "object", "properties": {"namespace": {"type":
61                  "string"}}}},
62                 {"name": "get_pod_logs", "description": "Fetch logs.",
63                  "inputSchema": {"type": "object", "required": ["pod_name"], "
64                  properties": {"pod_name": {"type": "string"}}}},
65                 {"name": "restart_deployment", "description": "Trigger
66                  restart.", "inputSchema": {"type": "object", "required": ["name"], "
67                  properties": {"name": {"type": "string"}}}}
68             ]}}
69         elif method == "tools/call":
70             name = params.get("name")
71             args = params.get("arguments", {})
72             res = TOOLS[name](**args) if name in TOOLS else "Error"
73             resp = {"jsonrpc": "2.0", "id": req_id, "result": {"
74                 "content": [{"type": "text", "text": res}]}
75             sys.stdout.write(json.dumps(resp) + "\n")
76             sys.stdout.flush()
77         except Exception as e:
78             pass
79
80 if __name__ == "__main__":
81     main()

```

Listing A.2 – Serveur MCP JSON-RPC Kubernetes sur stdio

## A.3 Annexe C : Manifeste GitOps ArgoCD ('argocd-app.yaml')

```

1 apiVersion: argoproj.io/v1alpha1
2 kind: Application
3 metadata:
4   name: wokmaster-gitops
5   namespace: argocd
6 spec:
7   project: default
8   source:
9     repoURL: 'https://github.com/farahbenchikha/kubernetes-enterprise-
10     platform.git'
11     targetRevision: main
12     path: kubernetes
13 destination:
14   server: 'https://kubernetes.default.svc'

```

```
14     namespace: default
15   syncPolicy:
16     automated:
17       prune: true
18       selfHeal: true
```

Listing A.3 – Manifeste d’application GitOps ArgoCD

---

# Webographie et Ressources

1. Kubernetes Documentation : <https://kubernetes.io/docs/>
2. HashiCorp Vault Injector : <https://developer.hashicorp.com/vault/docs/platform/k8s/injector>
3. Elastic Stack (ELK 8.x) : <https://www.elastic.co/guide/index.html>
4. ArgoCD GitOps : <https://argo-cd.readthedocs.io/>
5. K8sGPT CNCF Project : <https://k8sgpt.ai/>
6. Model Context Protocol (MCP) : <https://modelcontextprotocol.io/>
7. Google Gemini API : <https://ai.google.dev/docs>